Programming II

Error Handling with Exceptions

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

Exception Mechanism

Anomalous situations

Even the best code may suffer from anomalous scenarios leading to errors

- Operating system errors
- Resource unavailability
- User input errors
- Unexpected behaviors

Such errors should be spotted and correctly handled

- Stop execution
- Call error handling procedures
- ...

Error handling

```
float division(int num, int den) {
  float res;

  if (num != 0)              // this solves the problem
    res = num / den;
  else
    res = Float.MAX_VALUE;   // special value returned

  return res;
}
```

This fix make the code not reusable

- The caller may not spot the anomalous situation

Error handling (cont'd)

A special value can be returned to the caller

- Developers must remember error values
- Sometimes the error value is indeed a legitimate result
- Not suitable for constructors

Things are even more complex with nested calls

- Error location is difficult to track
- Or when the callee is not able to handle the error

A programmer burden

Without a specific language feature, error handling is up to the programmer

- The code will be messier

```
if (someFunc() == ERROR) { // the callee detected the error
  // the caller handles the error
} else {
  // normal execution
}
```

Only the direct caller can intercept the error (no upward delegation)

- Unless other boilerplate code is added

Exceptions

Anomalous situations happening during execution are called exceptions

- If not properly handled, exceptions may lead to a crash
- When something unexpected happens, some repairing code is executed

Java provides `try .. catch` blocks to programmatically handle exceptions

- Together with specific `Exception` classes

Handling exceptions

Java exception handling

Try	Code block where anomalous situations may happen
Throw	The exception(s) raised when anomalous situations happen
Catch	Code black that handles an anomalous situation

Workflow

- The program tries to execute some code (`try`)
- If something anomalous happens, the program raises an exception (`throw`)
- The exception can be caught and some repairing code executed (`catch`)

Handling exceptions (cont'd)

```
int readFile (String fileName) {
  // open the file
  if (operationError)
    return -1;
  // compute file size
  if (operationError)
    return -2;
  // allocate memory for file
  if (operationError)
    return -3;
  // read file into memory
  if (operationError)
    return -4;
  // close the file
  if (operationError)
    return -5;
  return 0; // everything went well
}
```

```
try {
  // open the file
  // compute file size
  // allocate memory for file
  // read file into memory
  // close the file
} catch (fileOpenError) {
  // handle file open error
} catch (fileSizeError) {
  // handle file size error
} catch (allocationError) {
  // handle allocation error
} catch (fileReadError) {
  // handle file read error
} catch (fileCloseError) {
  // handle file close
}
// everything went well
```

Business logic code decoupled from error handling code

Rising exceptions

Exceptions may be thrown by system or custom libraries

- The `FileNotFoundException` thrown by the `java.io.FileInputStream` class
- User code must handle them

Exceptions can be thrown by user code

- Already defined exceptions (e.g., `FileNotFoundException`)
- User-defined exceptions, extending the `Exception` class

Exception generation

1. Declare an exception class
2. Mark the method raising the exception
3. Instantiate the exception
4. Throw up the exception

```
public class EmptyStackException extends Exception { } // 1.

class Stack {
  public Object pop() throws EmptyStackException { // 2.
    if (stackSize == 0) {
      Exception e = new EmptyStackException(); // 3.
      throw e;  // 4.
    }
    ...
  }
}
```

Exception generation (cont'd)

Methods throwing exceptions must declare them after the `throws` keyword

```
public Object pop () throws EmptyStackException, IOException { ... }
```

The method must declare

- Exceptions directly thrown by the method
- Exceptions thrown by called methods which are not caught by the method

The execution of a method is interrupted when `throw` executes

Intercepting exceptions

Exceptions are caught when in the scope of a `catch` block

```
try {
  stack.pop();
  // code to execute when everything is ok
} catch (EmptyStackException e) {
  // handle error
  System.out.println(e);
}
```

Multiple `catch` block can be attached to a `try` block

Execution flow

Only one `catch` block is executed

```
File f new File("foo.txt");
try {
  f.open(); // throws FileError exception
  f.read();
  f.close();
} catch (FileError fe) {
  System.out.println("File error");
} catch (IOError ioe) {
  System.out.println("I/O error");
}
// continue execution
```

Execution flow

Only one `catch` block is executed

```
File f new File("foo.txt");
try {
  f.open();
  f.read(); // throws IOError exception
  f.close();
} catch (FileError fe) {
  System.out.println("File error");
} catch (IOError ioe) {
  System.out.println("I/O error");
}
// continue execution
```

Custom exception

In the constructor we can customize the exception behavior

```
public class EmptyStackException extends Exception {
  public EmptyStackException(Stack stack) {
    super("Pop␣Failure␣[stack␣'" + stack.toString() + "'␣empty]");
  }
}

class Stack {
  public Object pop() throws EmptyStackException {
    if (stackSize == 0) {
      throw new EmptyStackException(this);
    }
    ...
  }
}
```

Catching exceptions

When calling code possibly raising an exception, the caller can:

- Directly catch the exception
- Propagate the exception to the caller
- Catch the exception and re-throw it

```
public class Dummy {                                                    1
  public void foo() {                                                   2
    try {  // the line below may throw FileNotFound exception           3
      FileReader f = new FileReader("file.txt");                        4
    } catch (FileNotFound fnf) {                                        5
      // catch the exception and do something                           6
    }                                                                   7
  }                                                                     8
}                                                                       9
```

Catching exceptions (cont'd)

Exception can be ignored by the caller and propagated upwards

- Uncaught exceptions are propagated until `main()` and the JVM

```
public class Dummy {
  public void foo() throws FileNotFound {
      // here the handling of the exception is delegated to the caller
    FileReader f = new FileReader("file.txt");
  }
}

public class MyClass {
  public static void main(String[] args) throws FileNotFound {
    Dummy d = new Dummy();
    d.foo(); // this may throw FileNotFound exception
  }
}
```

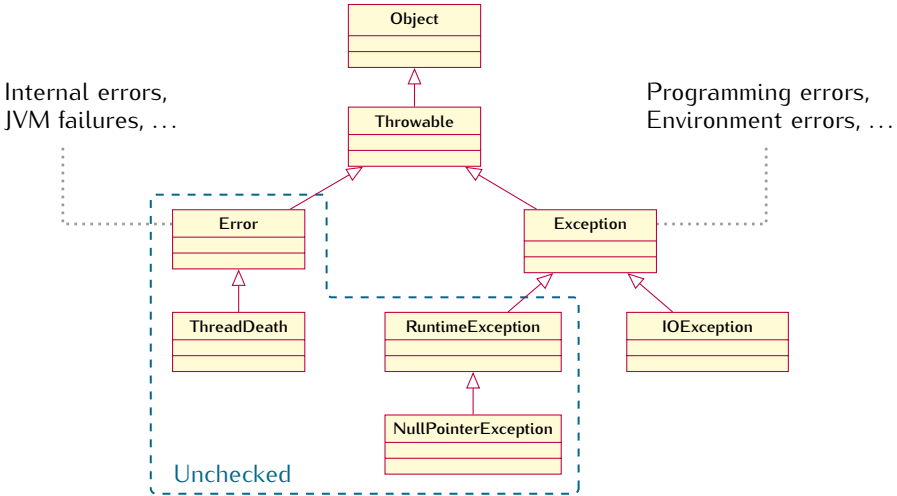
Catching exceptions (cont'd)

We can still propagate up locally caught exceptions

```
public class Dummy {                                                    1
  public void foo() throws FileNotFound {                               2
    try {  // the line below may throw FileNotFound exception           3
      FileReader f = new FileReader("file.txt");                        4
    } catch (FileNotFound fnf) {                                        5
      // catch the exception and do something                           6
      throw fnf; // and re-throw it (propagate to the caller)           7
    }                                                                   8
  }                                                                     9
}                                                                       10
```

Hierarchy of Exceptions

Exception class hierarchy



Checked and unchecked exceptions

Checked excpetions correspond to anomalous situations we can foresee

- Classic example: `FileNotFoundException`
- They must be declared, and thus checked by the compiler
- They are generated by user code with `throw`
- They must be caught by the caller

Unchecked excpetions correspond to anomalous situations we cannot foresee

- Classic example: `NullPointerException`
- They need not to be declared, and thus not checked by the compiler
- They are typically generated by the JVM
- They need not to be caught by the caller

Checked and unchecked exceptions (cont'd)

Checked exceptions

- ⊕ Who uses a method can immediately realizes what exceptions are possibly raised
- ⊕ The compiler automatically tells us if an exception is uncaught
- ⊕ The program will never halt at runtime, since exceptions are forced to be handled
- ⊖ Complicate method signatures

Checked exceptions

- ⊖ Who uses a method should inspect the method body in detail to look for exceptions
- ⊖ No support from the compiler on exception handling
- ⊖ The program may halt at runtime, since an exception may not have been handled
- ⊕ Method signature is unchanged

Which kind of exception to use

Checked exceptions propagate like a virus

- They must be recursively declared on all caller contexts
- Or caught at some point

Even if more secure, they can make the code almost unreadable

- The advice is to not define checked exceptions, unless strictly necessary

To define an unchecked exception

- Simply extend `RuntimeException` instead of `Exception`

Exception dirty tricks

Catch a checked exception and re-throw it as an unchecked exception

```
public class MyException extends Exceptions { } // checked

public void foo() throws MyException {
  throw new MyException();
}

public void bar() throws MyException { // mandatory declaration

    foo();



}
```

Exception dirty tricks

Catch a checked exception and re-throw it as an unchecked exception

```
public class MyException extends Exceptions { } // checked

public void foo() throws MyException {
  throw new MyException();
}

public void bar() { // no declaration needed
  try {
    foo();
  } catch (MyException e) {
    throw new RuntimeException(e); // unchecked
  }
}
```

Exceptions and loops

For errors affecting single iterations, the `try .. catch` block is nested in the loop

- In case of an exception, proceed with the next iteration

```
while (true) {
  try {
    // code potentially rising an exception
  } catch (AnException e) {
    // handle local error
  }
}
```

Exceptions and loops (cont'd)

For errors compromising the whole loop, the `try .. catch` block wraps the loop

- In case of an exception, exit the loop

```
try {
  while (true) {
    // code potentially rising an exception
  }
} catch (AnException e) {
  // handle global error
}
```

Q + A